Lecture 01: Distributed Systems, Virtualisation and the Cloud (and module introduction)

COMP2014 Distributed Systems

Chris Greenhalgh

School of Computer Science

Errata: Friday lecture location corrected (2026-01-29)

Notices to share with students at the start of an EchoPoll session

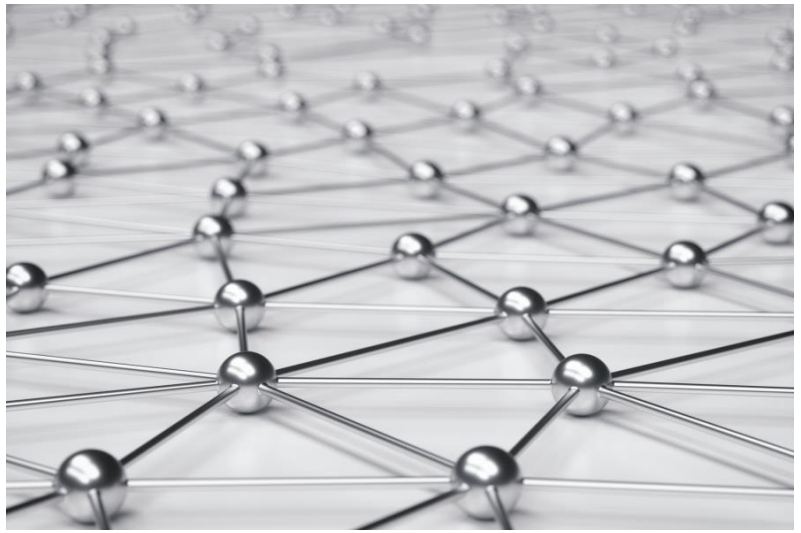
The following notices are taken from the 'Code of Practice for the Safe Use of EchoPoll v1.1'

While text responses may sometimes appear to be anonymous to the audience, they are always traceable

Please be advised that names and e-mail addresses of respondents may be displayed on screen

Contents

- Module information
- Distributed Systems
- Virtualisation and Containers
  - The Cloud
  - SSH



Part 1: Module Information

Education Aims (module catalogue)

“To teach students about

- the nature and applications of distributed systems,
- how distributed systems are constructed and
- the key characteristics and challenges of distributed systems.”

Summary of Contents (module catalogue)

“This module covers the following topics:

- **overview** of parallel and distributed computing;
- **applications** of distributed systems;
- **fundamental concepts** of distributed systems (processes and message passing, naming and discovery, fault tolerance and partial failure, consistency and cacheing, security);
- [reliable network communication];
- distributed system **design approaches** (direct vs indirect communication, client-server vs peer-to-peer, stateful vs stateless interfaces);
- introduction to **distributed data management**;
- introduction to **distributed algorithms**.”

Weekly plan (provisional)

Week 1: (inc. docker lab)

- Distributed Systems, Virtualisation
- Distributed Systems & Networking (1, inc. UDP)

Week 2: (inc. TCP/UDP lab)

- Distributed Systems & Networking (2, inc. TCP)
- Network Security

Week 3: (inc. RMI lab)

- RPC and RMI
- DNS

Week 4: (inc. DNS lab)

- Q&A, revision
- WWW

Week 5: (inc. WWW lab)

- HTTP servers
- HTTP APIs

Week 6: (inc. Java WWW lab)

- Physical & architectural models
- Indirect Comm'n: Group Comm'n and Pub-Sub

Week 7: (inc. Redis lab)

- Indirect Comm'n: Message Queues and Tuple Spaces
- Q&A, revision

Week 8:

- Overlay network and P2P
- Multicast & discovery

Week 9: (inc. Multicast lab)

- Distributed algorithms
- Failure & reliable communication

Week 10:

- (More) Data & Failure
- Q&A, revision

Week 11:

- Revision / Q&A

Convenor

- Prof. Chris Greenhalgh
  - https://www.nottingham.ac.uk/computerscience/people/chris.greenhalgh
  - Mixed Reality Lab
    - HCI; collaborative system; ubicomp; music; mental health/wellbeing
- Contact:
  - **Email**: chris.greenhalgh@nottingham.ac.uk
    - Default way to ask questions or make suggestions outside labs & lectures (if questions are general I'll post the response as an announcement...)
  - Computer Science, room C5 – but make an appointment if possible
  - (Teams – note pre-arranged video/audio calls only. Also note, email is more reliable than Teams chat)

Resources

- Module page on Moodle, https://moodle.nottingham.ac.uk/course/view.php?id=158551 with:
  - Announcements
  - Previous years' exam paper & solutions
  - Weekly: Lecture slides & videos; Lab resources (see later slide); exercises
  - Readling list, https://rl.talis.com/3/notts/lists/A1D9A07C-F6A9-A3DE-AC62-87F2012334D0.html ...

Resources (cont.)

- Text book: *Distributed systems: concepts and design*, Coulouris et al (5th ed.)
  - Optional, £40-55; has more/deeper coverage in most parts (physical and e-copies – see link to reading list in Moodle)
- See also: "Distributed Systems", Van Steen; & Tanenbaum (third edition)
  - Further background reading; free (personalised) e-book download https://www.distributed-systems.net/index.php/books/ds3/ds3-ebook/
- And a couple of potentially useful books on architecture and one on REST APIs – you can find more in https://nusearch.nottingham.ac.uk/

Labs

- Chance to explore topics and issues through hands-on examples
  - Not directly assessed, but help to build skills, ground and illustrate lecture material and help with exam questions
- ~8 topics
  - Docker; Java TCP & UDP; Java RMI; DNS; HTTP; Java HTTP; Redis; Java Multicast
  - “Worksheet” for each lab and a sample solution
- **It is IMPORTANT that you do the labs. They WILL help you understand the material in the exam.**

Delivery (2025/26)

- In-person lectures
  - Normally recorded, with videos available afterwards
  - Tuesday 9am, JC BS-South A25
  - Friday 9am, ~~JC BS-South A25~~ JC Exchange B01
- Lab sessions:
  - Thursday 4pm [Preferred!] OR 5pm [if required], JC Dearing A34
- **Note:** see detailed schedule on Moodle; any changes to lectures and labs will be notified via the news/announcements on Moodle

Assessment

- In-person 2 hour unseen written exam
  - Exact exam format essentially the same as 2024/25
    - all questions compulsory
    - (Note, the scenario(s) used are different for every exam :)
- Past paper questions
  - Last three years' papers and sample solutions are available on moodle, with general feedback

Q: questions or concerns?

Prerequisites

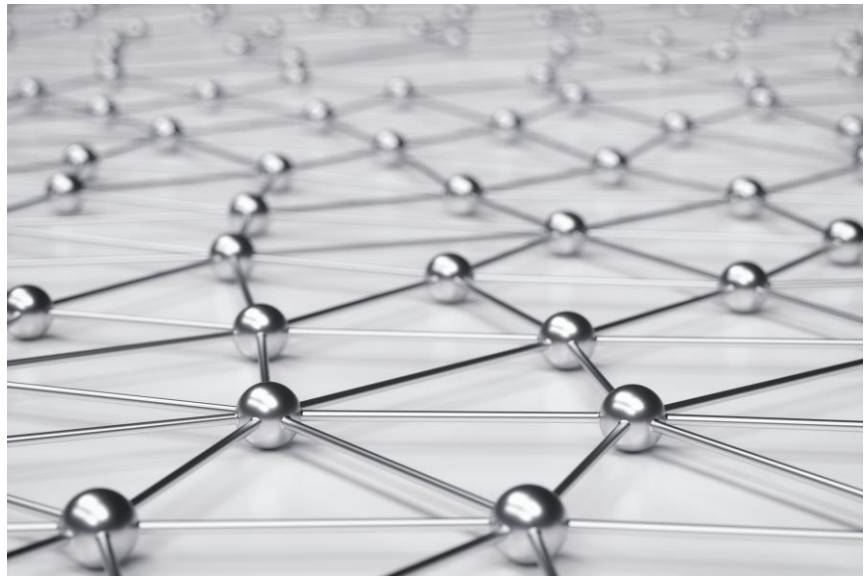
- Target students:
  - Part I undergraduate students in the School of Computer Science.
  - An option on the Operating Systems and Networks theme
- Pre-requisites:
  - Some knowledge of basic computer architecture, networking and concurrency (threads)
  - Ability to use command line (esp. Linux)
  - Ability to read/understand Java programs (examples) and write simple Java interfaces
  - Ability to use git (at least to check out repositories) and VSCode

Distributed Systems

Definition

- *"We define a distributed system as one in which hardware or software **components** located at **networked computers** communicate and coordinate their actions only by passing **messages**."*
  - Distributed Systems: Concepts and Design (Edn. 5), Coulouris, Dollimore, Kindberg and Blair

- Distributed systems are everywhere!



Resource sharing

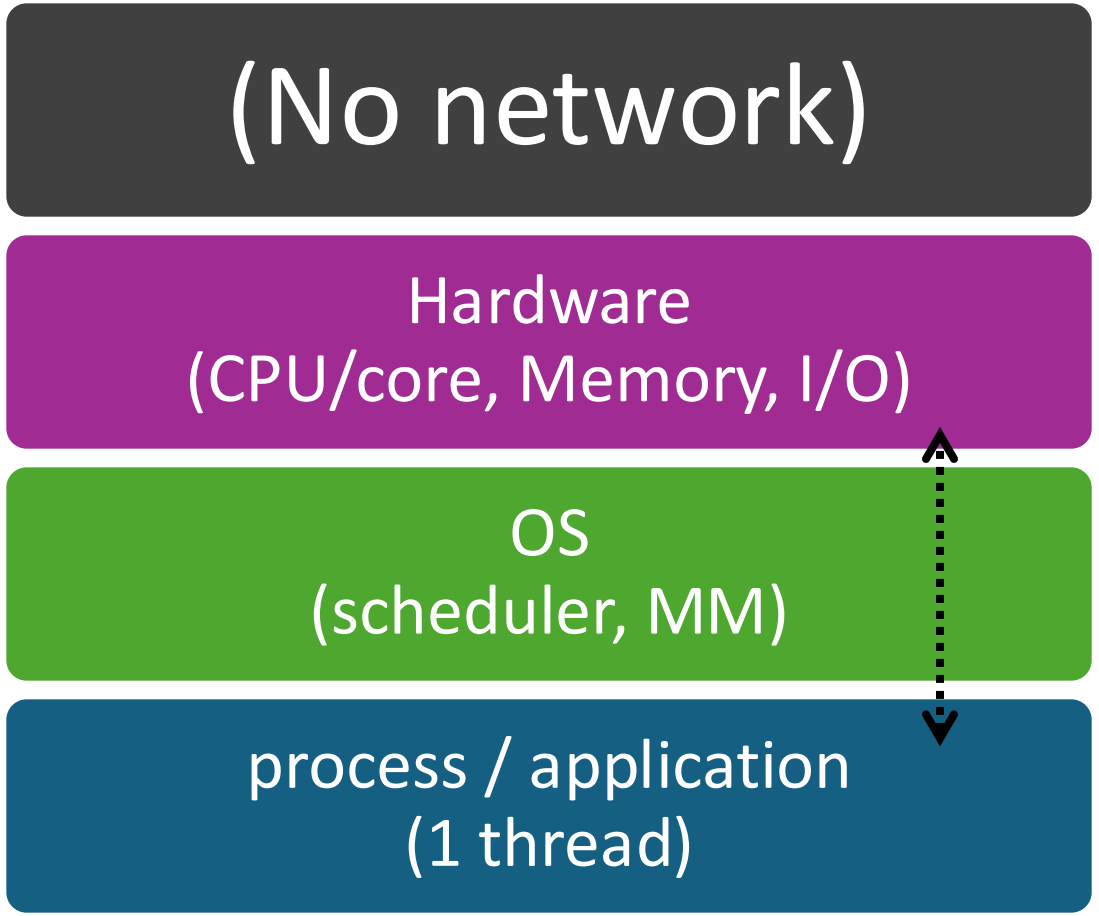
- Most applications of distributed systems are based on **sharing resources**
  - (or explicit communication and/or coordination)
- E.g.
  - **Physical** resources, such as printers, disks, computers
  - …and **People**! (i.e. communication / coordination)
  - **Data** resources, such as files, documents, databases, web pages
  - **Computational/algorithmic** resources, such as search engines or machine-learning algorithms

Example services & associated resources

- Remote access (e.g. SSH, RDP)
  - Particular shared computer (CPU, memory, files, programs)
- Network file storage (e.g. NFS, OneDrive)
  - Persistent (disk) storage
- Authentication (e.g. Active Directory, LDAP)
  - Accounts and credentials (e.g. passwords)
- Virtual desktop (e.g. Windows Virtual Desktop)
  - On-demand (Windows/Linux) computing/desktop environment
- Office 365
  - Word-processor/document, etc.
- Moodle
  - Learning resources, assignments, ...
- The World Wide Web
  - “Resources”, e.g. documents, web pages
- Teams, zoom, skype, …
  - Other people (?!) / remote mics, speakers, cameras, screens
- Anything “Cloud” …
  - On-demand compute, storage, communication, platforms, applications, …

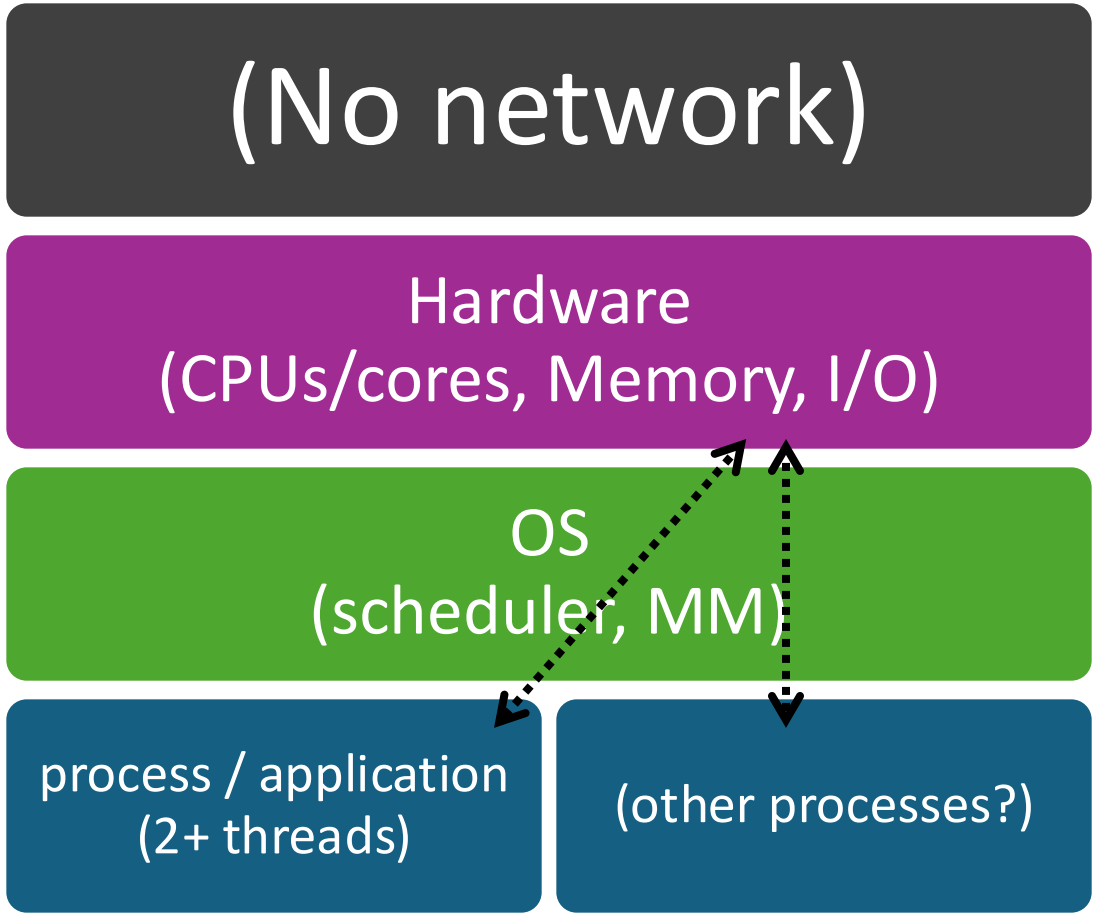
Sequential system

- 1 computer
- 1 thread => 1 instruction sequence
- Memory
  - i.e. variables, etc.
- (No network required)



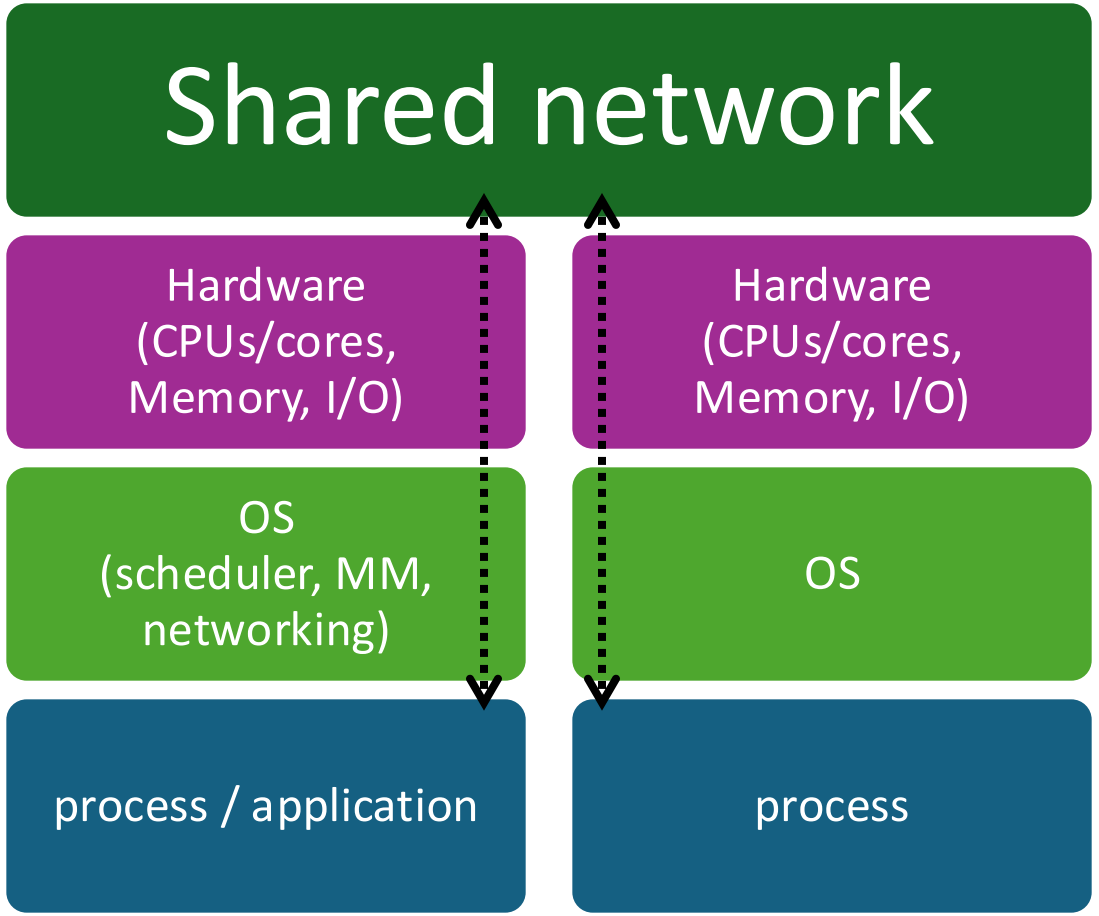
Parallel system

- 1 computer
- 2+ threads
  => 2+ instruction sequences
  - (Multiple cores/CPUs => physical speedup)
- **Common physical memory**
  - For communication & coordination
  - i.e. **SHARED memory / variables**
    - & locks & semaphores, etc.
- (No network required)



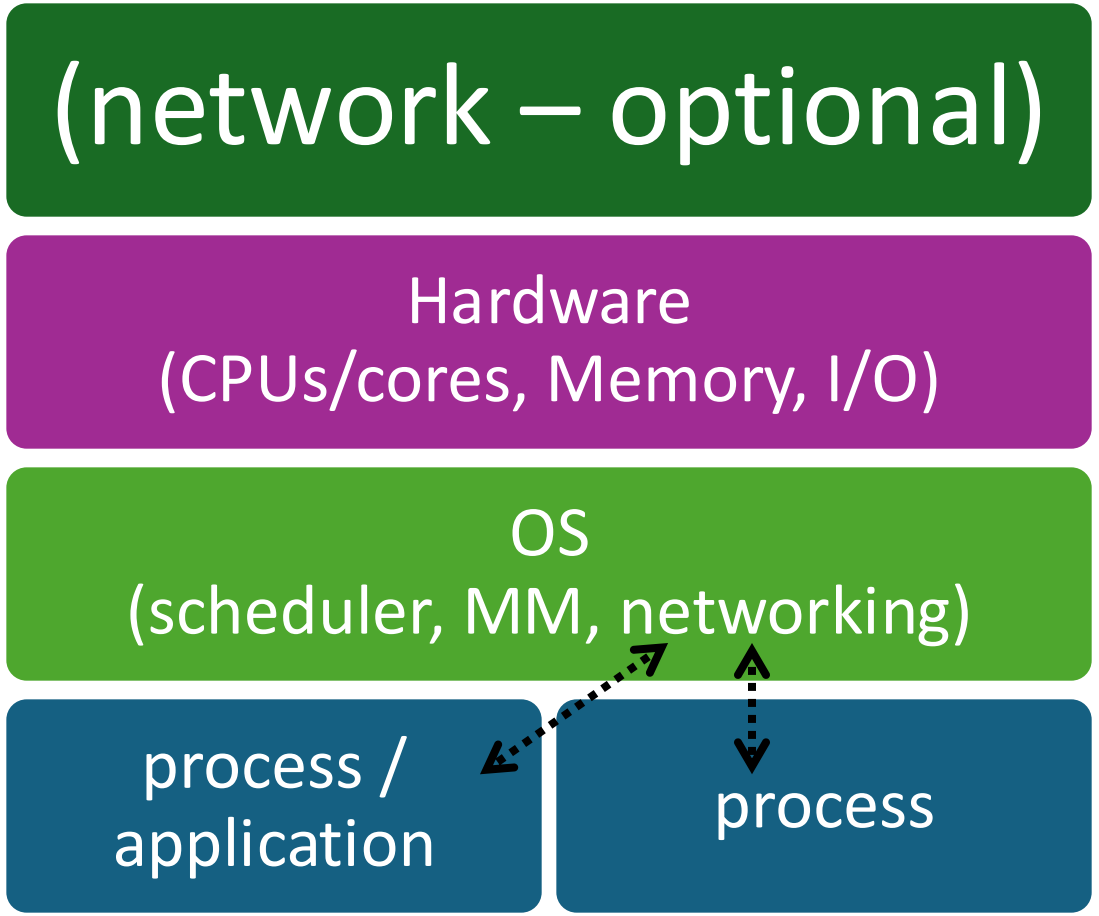
Distributed system

- 2+ computers
- ≥1 process / thread each
  => 2+ instruction sequences
- NO common physical memory
  - i.e. NO shared variables
    - NO locks, semaphores, etc.
- **Common physical network**
  - For communication & coordination
  - **Message passing**



But one computer can effectively be a distributed system…

- 1 Computer
- 2+ processes / threads
  => 2+ instruction sequence
- Not USING common memory
- **Common virtual (OS) network**
  - For communication & coordination
  - E.g. “Loopback” device
    - = “localhost” name
  - **Message passing**

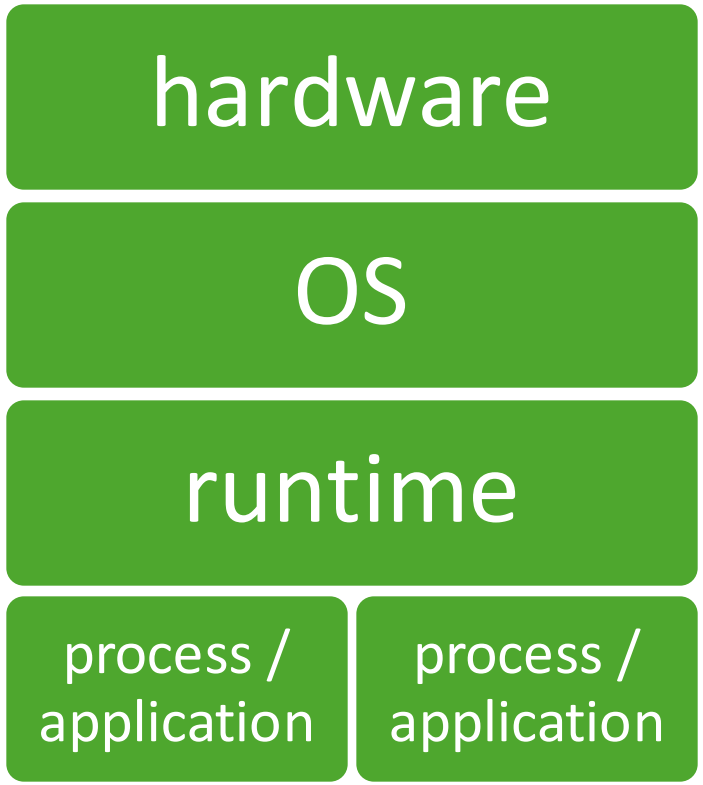


Virtual machines, containers, the cloud and SSH

Machines, Virtual Machines & Containers

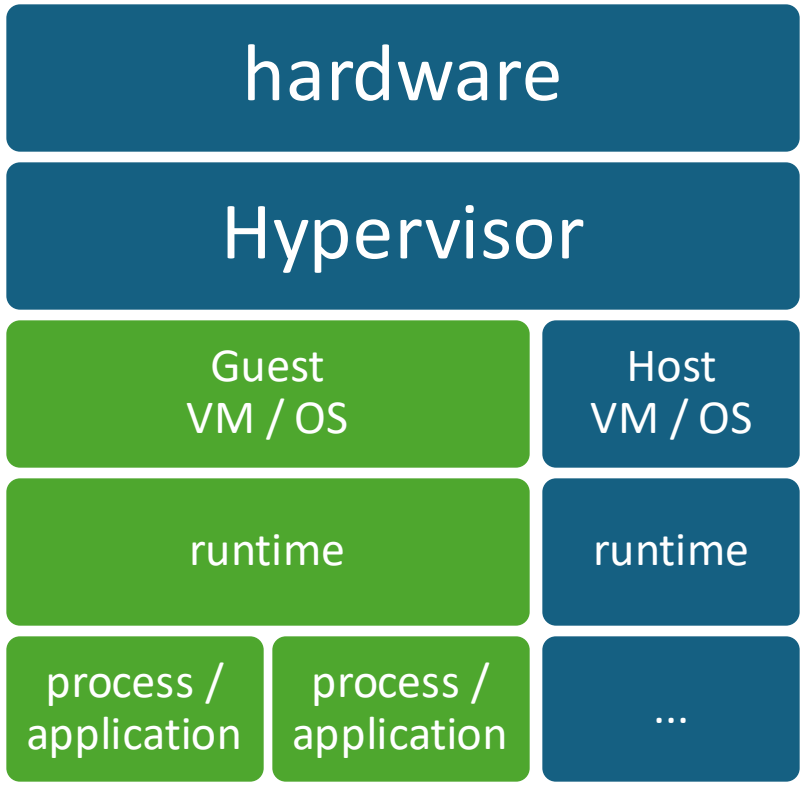
Q: Used containers?

Regular physical machine:
Single OS, Single runtime,
runs multiple processes



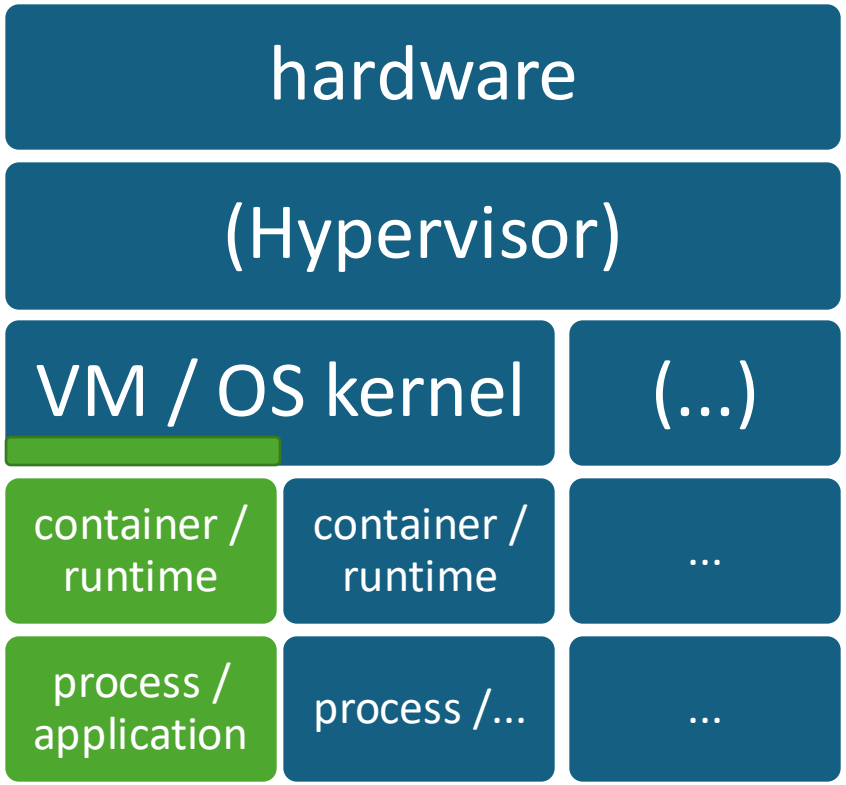
(Note: an emulator creates a virtual machine in software and is generally more flexible but much slower) -->

Virtual Machine: sharing hardware,
each VM with its OS & runtime,
runs multiple processes
e.g. VMWare, VirtualBox, HyperV



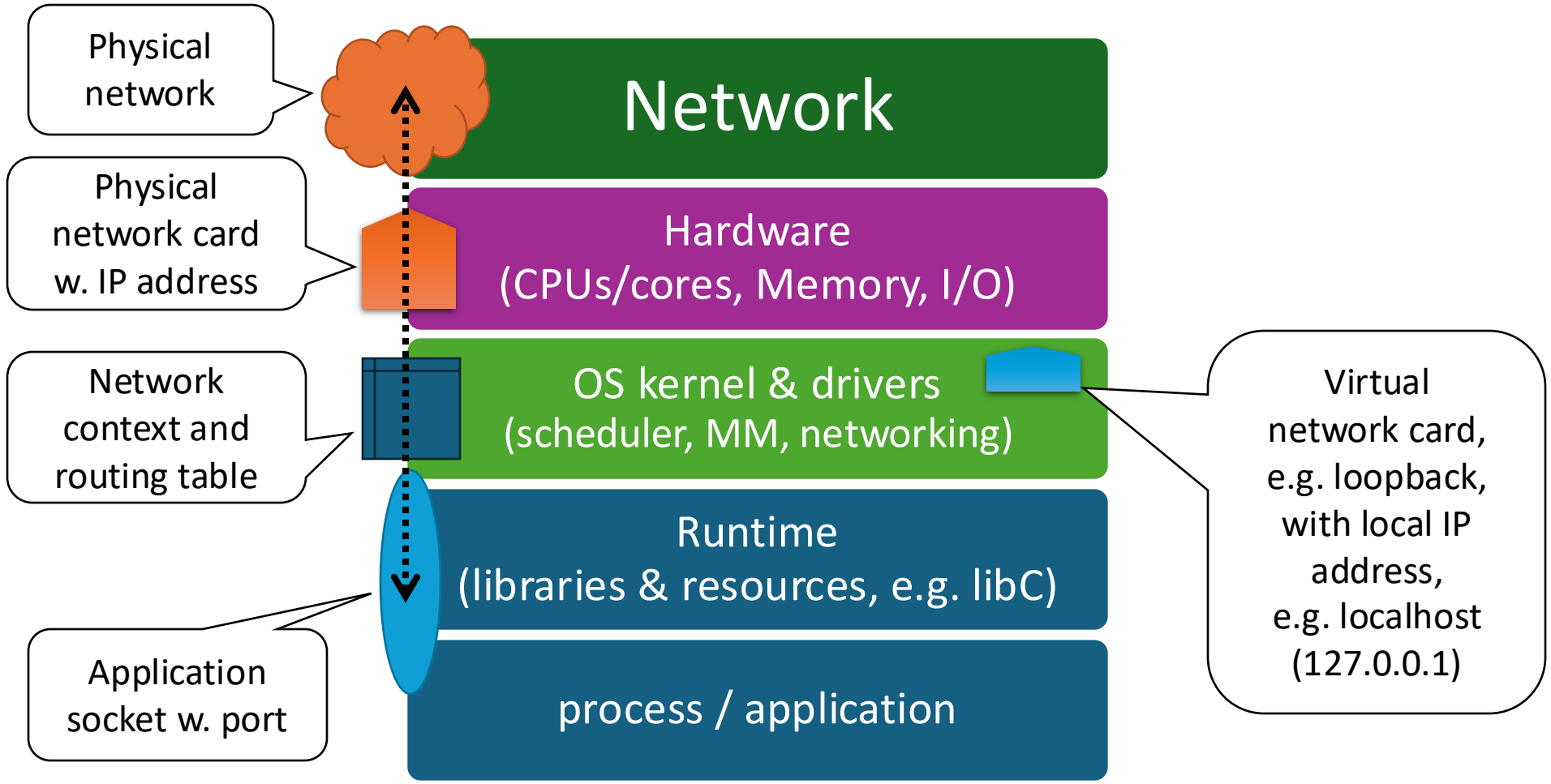
NB: a (hypervisor-based) VM is as fast as native hardware, and can be created and managed dynamically via an API

Container: sharing OS (kernel),
each container runs one [main] process
with its own runtime
e.g. Docker, rkt, lxc

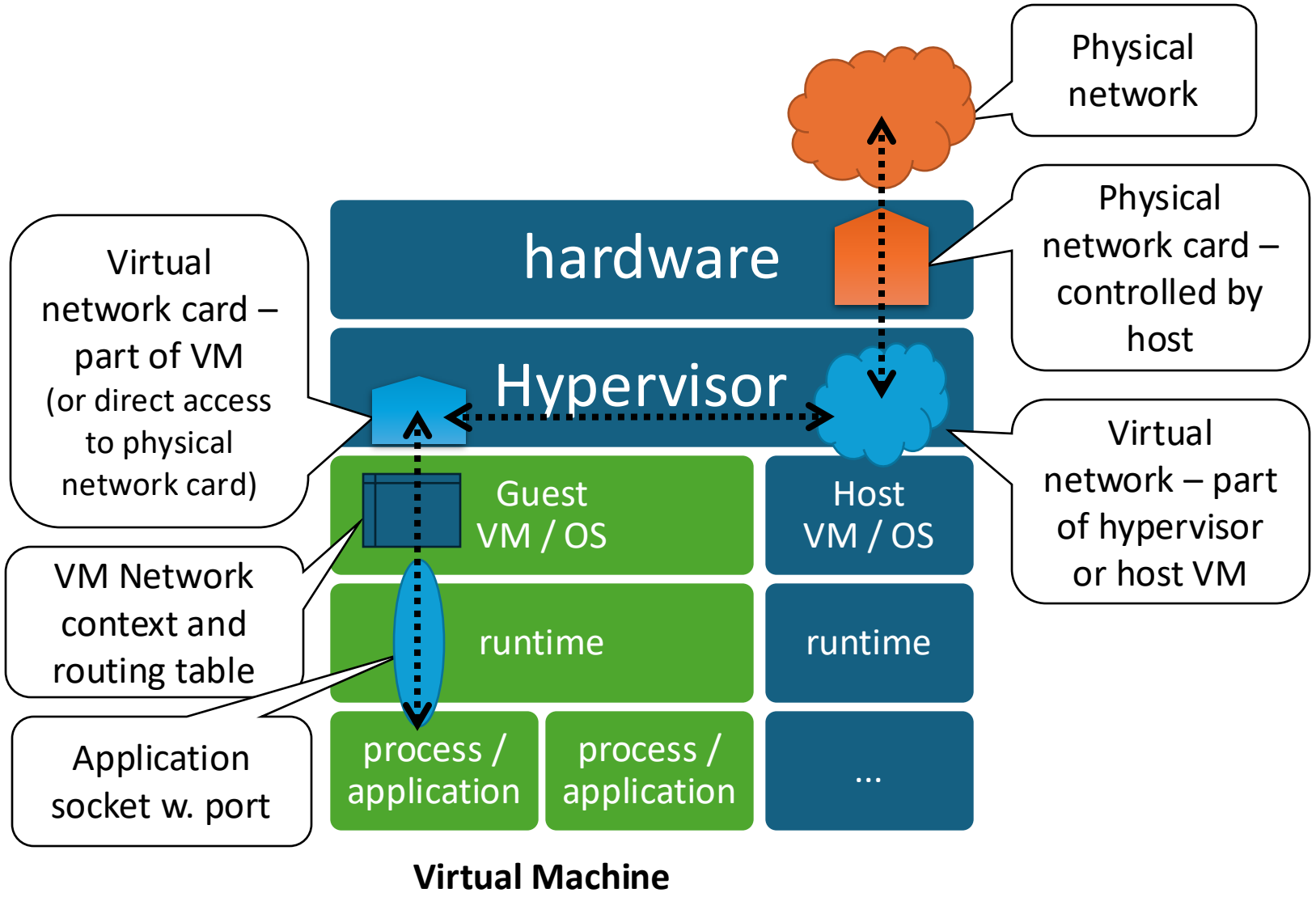


NB: a container is self-contained, portable and lower overhead than a full VM, with lots of supporting tools

A networked computer in a little more detail…



Virtual Machine Networking Components

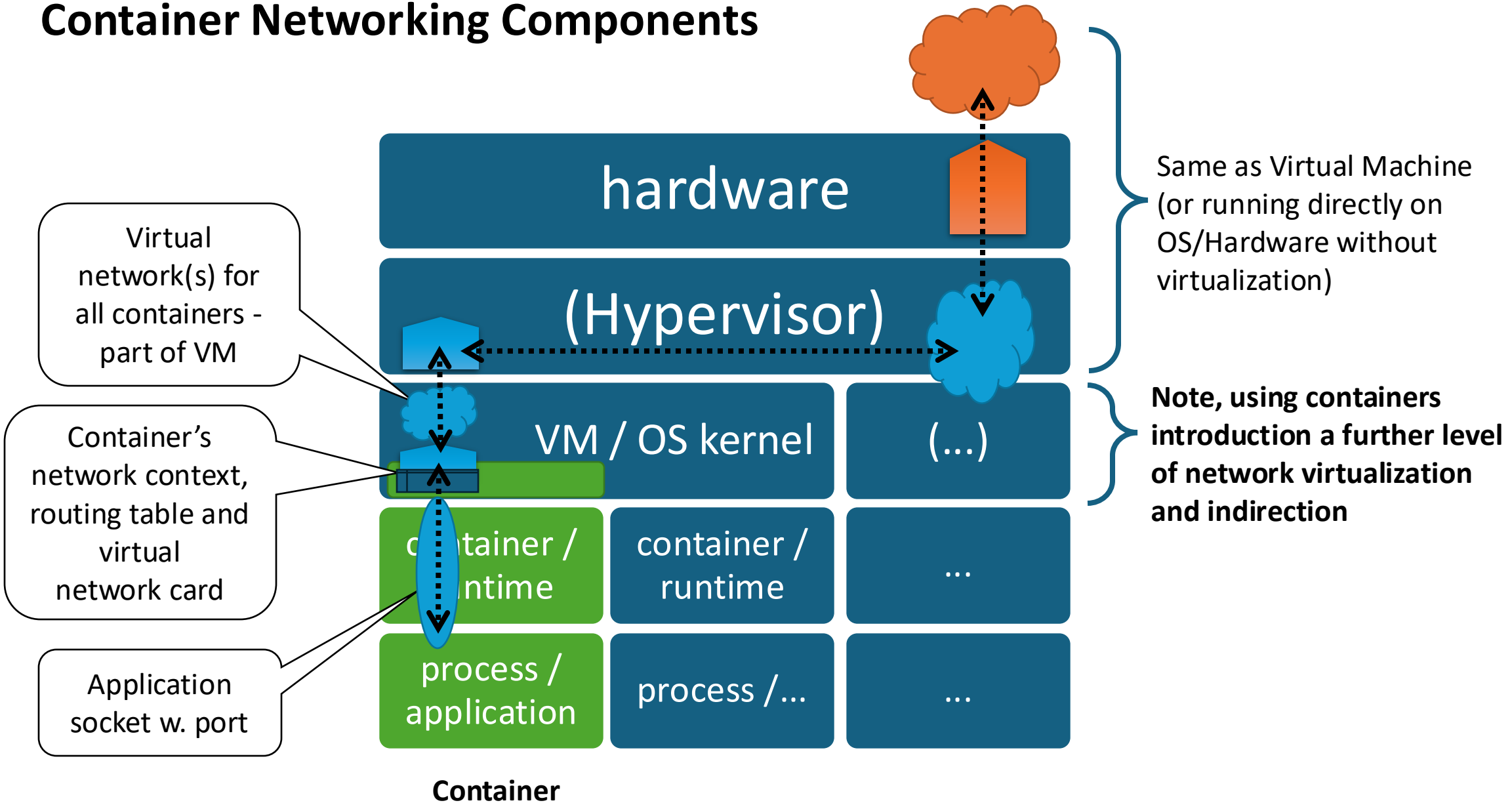


EVERY network interface has:
IP address, network address range
(State – up or down)
(Multicast capable or not)

And **every OS & container** has:
A **routing table** with a default route

(typically) Application port and IP are **private** and selectively mapped to external IP/port w. **Network Address Translation (NAT)**, allowing outgoing connections but requiring explicit mappings for incoming connections

Container Networking Components



Q: Used cloud services? (other than SaaS)

The Cloud

- = **software-managed**, **on-demand**, **pay-as-you-go** computing in remote datacentres, accessed over the internet, enabled by virtualization
  - e.g. Microsoft Azure, Google Cloud Platform, Amazon Web Services
- Many different kinds of services available, e.g.
  - **Infrastructure as a Service (IaaS) = Virtual machines, virtual disks, etc.**
  - Containers as a Service (CaaS) = Container (i.e. containerized application) hosting, e.g. Azure Kubernetes service
  - Platform as a Service (PaaS) = Language-specific application hosting, e.g. Azure Web Apps, PHP hosting, Azure Databricks (Apache Spark)
  - Serverless = Autoscaling application or function hosting, e.g. Azure Functions
  - Other managed applications, e.g. Azure SQL Database, Azure Event Hub
  - Software as a Service = end-user applications, generally browser-based (e.g. Office365)
  - Desktop as a Service (DaaS), e.g. Windows Virtual Desktop

SSH (Secure SHell)

(in case you need it)

- SSH is a network protocol allowing **secure remote access** to (esp. Unix/Linux) hosts over TCP
- A SSH server accepts and authenticates connections (by default on port 22) against local users
  - E.g. using a username and password or pre-configured private SSH key
- If successful, it creates an **interactive shell** session running as that user
  - i.e. a remote terminal or command window (just text)
- Useful SSH clients include:
  - OpenSSH – often run from the (local) command line as “ssh”
  - VSCode, with the “Remote Development” extension pack
  - Ansible, for automating system admin
- Windows-specific clients include:
  - SSH Secure Shell Client – but the UoN version of is VERY old and out of date
  - PuTTY
- Can also be used for:
  - Copying files (sftp, e.g. psftp)
  - Forwarding TCP connections (port forwarding)
  - ”Jumping” or tunnelling to another host, e.g. CS SSH Bastions

Summary: Distributed Systems and the Client-Server Model

- A **distributed system** is one in which “hardware or software components located at **networked computers** communicate and coordinate their actions only by passing **messages**.”
- Typically distributed systems are built around **sharing of resources**, whether physical or informational
- The software components are typically OS **processes**,
- and communicate directly with each other using **Internet Protocol (IP)**

VM, Containers and the Cloud Summary (pt1)

- A **hypervisor** allows efficient virtualization of a single computer to create multiple isolated **Virtual Machines** (VM)
  - Each VM has its own operating system (kernel and runtime) and can run multiple processes (applications)
  - (A pure emulator, e.g. QEMU without KVM, can also create VMs and is more flexible, i.e. can emulate a different CPU, but is much less efficient)
- A single machine (physical or virtual) can run multiple **containers**, which share the same OS kernel but have a separate runtime and (generally) each run one application
  - (Note, VS-Code Dev Containers uses a live container as a context in which to **develop** - not just run - an application in a programming language)

VM, Containers and the Cloud Summary (pt2)

- Each VM or container (typically) has it's own network context, routing table, **virtual network interface** and **IP address**
  - Usually a private IP on a local virtual network, often with selected ports exposed through **Network Address Translation**
  - Sometimes directly visible on the physical network, e.g. if configured to use the host network interface directly, or on cloud VMs with public IPs
- The **Cloud** = on-demand, pay-as-you-go remote computing
- **SSH** (Secure Shell) is the standard protocol for secure remote terminal access to (esp.) Linux/Unix hosts (including VMs).

Next steps

- Lab 1 – containers & docker (with SSH & VMs if required)
  - Thursday 4pm/5pm, Dearing A34
- Lecture 2 – Networking (recap) & Java TCP Programming
  - Friday 9am
- Look through the moodle resources, e.g. past papers